



TAREFA

Unidade 6 | Capítulo 1

Infraestrutura como Código (IaC) com Terraform e AWS

Projeto: Desenvolvimento de Projeto AVANÇADO

Autora: Marcela Monteiro Gondim

Repositório GitHub: [MMGondim/projeto-capacita-aws](https://github.com/MMGondim/projeto-capacita-aws)

REALIZAÇÃO



EXECUÇÃO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



1. Introdução

O que é Infraestrutura como Código (IaC)?

Tradicionalmente, a criação de servidores, redes e permissões na nuvem era feita manualmente através de cliques em um painel web, como o AWS Management Console. No entanto, esse processo manual apresenta desvantagens:

- Lento e suscetível a erros humanos;
- Difícil de replicar com exatidão em múltiplos ambientes (Dev, Staging, Prod);
- Ausência de histórico e versionamento.

A IaC é a prática de gerenciar e provisionar servidores, redes e bancos de dados por meio de arquivos de configuração legíveis por máquina, substituindo os processos manuais e painéis físicos. Ela permite criar ambientes idênticos sob demanda de forma rápida e automatizada através da definição de servidores, filas, redes e políticas em arquivos de texto declarativos.

Porque usar o Terraform?

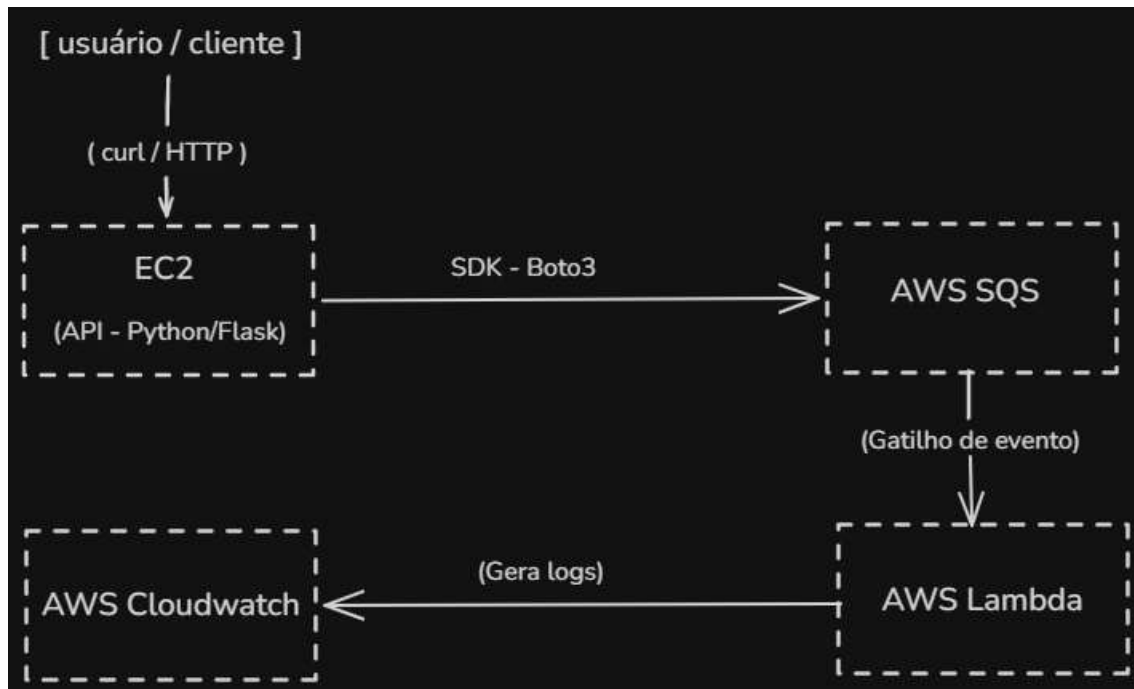
O Terraform (da HashiCorp) é uma ferramenta *multicloud* que permite declarar o estado desejado da arquitetura. Em vez de dizer como criar um recurso passo a passo (linguagem imperativa), é declarado o que é desejado (linguagem declarativa), e o Terraform calcula o plano de execução.

2. Conceitos Fundamentais

- HTTP: *Hypertext Transfer Protocol* (Protocolo de Transferência de Hipertexto). É o conjunto de regras usado para transferir dados na internet entre um cliente (como o seu navegador) e um servidor web. É a base para carregar páginas, imagens e vídeos na rede.
- CURL: *Client URL* (Cliente URL). É uma ferramenta de linha de comando utilizada para transferir dados via protocolos de rede, como o HTTP/HTTPS, simulando a ação de um cliente (como um navegador web, um aplicativo de celular ou outro servidor) enviando uma requisição.
- EC2: *Elastic Compute Cloud* (Nuvem de computação elástica). É o serviço da Amazon Web Services (AWS) que permite eliminar a necessidade de aquisição de hardware físicos através do aluguel de servidores virtuais em nuvem. Suas máquinas virtuais são chamadas de instâncias.
- API: *Application Programming Interface* (Interface de Programação de Aplicação). É um conjunto de regras e padrões que permite que diferentes sistemas de software conversem entre si e troquem dados de forma segura, sem que um precise saber os detalhes internos de funcionamento do outro.
- Flask: É estrutura de código pronta que funciona como um esqueleto ou uma caixa de ferramentas para criar software, ou seja, um *framework* de desenvolvimento web escrito em Python. Ele serve para criar sites, aplicações web e APIs de forma rápida, leve e flexível, fornecendo apenas o núcleo essencial.
- SDK: *Software Development Kit* (Pacote de desenvolvimento de software). É um conjunto de ferramentas e bibliotecas fornecido por uma plataforma para que desenvolvedores possam escrever código em sua linguagem preferida e interagir com os serviços dessa plataforma. O SDK simplifica essa comunicação através de funções nativas na linguagem escolhida.
- Boto3: É o SDK oficial da AWS para a linguagem Python. A grande vantagem do uso do Boto3 é que ele detecta automaticamente a IAM Role que foi anexada à EC2 - o Perfil da Instância- e faz a autenticação segura com a AWS sem precisar de senhas ou chaves gravadas no código.
- AWS SQS: *Simple Queue Service* (Serviço de fila simples). É um serviço de fila de mensagens distribuído e totalmente gerenciado.
- AWS Lambda: Serviço de computação *serverless* e orientada a eventos. O AWS Lambda permite executar código sem gerenciar servidores e com pagamento atrelado ao uso, pelo tempo de execução em milissegundos.
- AWS Cloudwatch: É o serviço de monitoramento e gerenciamento de logs da AWS.

3. A Arquitetura da Solução

Qual a lógica de negócio utilizada?



4. Detalhamento Passo a Passo da Infraestrutura

Passo 1: Rede e Isolamento (VPC, Subnet, *Internet Gateway*)

1. **Conceito:** A VPC (Nuvem Virtual Privada) é uma rede virtual privativa e isolada dentro da AWS que pode ser dividida em sub-redes. A Subnet Pública é um tipo de sub-rede que possui rota para a internet através de um *Internet Gateway*.
2. **O que foi feito:** Foi criada uma VPC personalizada e, em seguida, configurada a tabela de roteamento para direcionar o tráfego externo ao *Internet Gateway*.
3. **Por que/Para que:** Para garantir que a aplicação esteja isolada de outras redes na nuvem, mantendo o controle total sobre quais IP's (Protocolo de Internet) podem acessá-la, ao mesmo tempo que é permitido o acesso público HTTP à API utilizada.
4. **Arquivos:**
 1. provider.tf

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}
```

2. vpc.tf

```

resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true

  tags = {
    Name = "capacita-iredo-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "capacita-iredo-public-subnet"
  }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "capacita-iredo-igw"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}

resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

```

Passo 2: Segurança de Rede (*Security Group*)

- **Conceito:** Um *Security Group* (Grupo de Segurança) atua como um sistema de segurança que monitora, filtra e controla o tráfego de dados entre um computador ou rede interna e a internet - *firewall* - virtual estatal que controla o tráfego de entrada (*Inbound*) e saída (*Outbound*) das instâncias.
- **O que foi feito:** Foram liberadas explicitamente as portas:
 - 22 (SSH): Para acesso remoto via terminal ao servidor.
 - 5000: Porta personalizada onde a aplicação Web em Python/Flask escuta as requisições.
- **Por que/Para que:** Foi aplicado o Princípio do Menor Privilégio, ou seja, por padrão, todo o tráfego de entrada na AWS é bloqueado, liberando estritamente o necessário para o funcionamento da aplicação.
- **Arquivos:**
 - security_group.tf

```
resource "aws_security_group" "ec2_sg" {
  name        = "capacita-ired-e-ec2-sg"
  description = "Permite tráfego SSH e HTTP na porta 5000"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "API Flask"
    from_port   = 5000
    to_port     = 5000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Passo 3: Computação e Automação de Inicialização (EC2 & *User_Data*)

- **Conceito:** A Amazon EC2 provê capacidade computacional escalável na nuvem através de servidores virtuais. O *User_Data* é um script executado automaticamente no primeiro *boot* da máquina.
- **O que foi feito:** Foi provisionada uma instância t4g.micro - eficiente em custo e desempenho rodando Ubuntu. No *user_data*, foi automatizada a instalação do Python, dependências e a inicialização da API Flask.
- **Por que/Para que:** O *user_data* garante a imutabilidade e a reprodutibilidade do ambiente: se o servidor for destruído e recriado, ele subirá pronto para responder requisições sem nenhuma intervenção humana manual.
- **Arquivo:**
 - ec2.tf

```
resource "aws_instance" "app_server" {
  ami           = "ami-0c7217cddde317cfec" # Ubuntu Server ARM na us-east-1
  instance_type = "t4g.micro"
  subnet_id     = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.ec2_sg.id]
  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y python3-pip python3-venv

    mkdir -p /app
    cd /app

    python3 -m venv venv
    source venv/bin/activate
    pip install flask boto3 gunicorn

    cat << 'PYEOF' > app.py
    from flask import Flask, request, jsonify
    import boto3
    import os

    app = Flask(__name__)
    sqs = boto3.client('sqs', region_name='us-east-1')
    QUEUE_URL = "${aws_sqs_queue.pedidos_queue.id}"

    @app.route('/produtos', methods=['GET'])
    def get_produtos():
        return jsonify([
            {"id": 1, "nome": "Tablet", "preco": 1500},
            {"id": 2, "nome": "Fone Bluetooth", "preco": 200}
        ])
  >>>EOF
}
```



```

@app.route('/produtos', methods=['GET'])
def get_produtos():
    return jsonify([
        {"id": 1, "nome": "Tablet", "preco": 1500},
        {"id": 2, "nome": "Fone Bluetooth", "preco": 200}
    ])

@app.route('/pedidos', methods=['POST'])
def post_pedido():
    dados = request.get_json()
    response = sqs.send_message(
        QueueUrl=QUEUE_URL,
        MessageBody=str(dados)
    )
    return jsonify({
        "status": "Pedido enviado para o SQS com sucesso!",
        "message_id": response['MessageId']
    }), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
PYEOF

nohup python3 app.py > /app/app.log 2>&1 &
EOF

tags = {
    Name = "capacita-irede-ec2-api"
}

output "ec2_public_ip" {
    value      = aws_instance.app_server.public_ip
    description = "IP público da instância EC2"
}

```

Passo 4: Segurança e Acesso entre Serviços (IAM Instance Profile)

- **Conceito:** O AWS IAM (Gerenciamento de Identidade e Acesso) gerencia permissões de acesso. Um Perfil de Instância permite associar uma *Role* (papel com permissões) diretamente a uma instância EC2.
- **O que foi feito:** Foi criada uma IAM Role com permissão de escrita no SQS (sqs:SendMessage) e a atribuída à EC2.
- **Por que/Para que:** Evita colocar chaves de acesso (AWS_ACCESS_KEY_ID / SECRET) gravadas no código Python ou no servidor. A EC2 assume o papel temporário de forma transparente e segura via SDK (boto3).
- **Arquivo:**
 - iam.tf

```

resource "aws_iam_role" "ec2_sqs_role" {
  name = "capacita-irede-ec2-sqs-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy" "sqs_write_policy" {
  name = "sqs-write-policy"
  role = aws_iam_role.ec2_sqs_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = ["sqs:SendMessage"]
      Effect = "Allow"
      Resource = aws_sqs_queue.pedidos_queue.arn
    }]
  })
}

resource "aws_iam_instance_profile" "ec2_profile" {
  name = "capacita-irede-ec2-profile"
  role = aws_iam_role.ec2_sqs_role.name
}

```

Passo 5: Desacoplamento e Processamento Assíncrono (AWS SQS)

- **Conceito:** O AWS SQS é um serviço de fila de mensagens distribuído e totalmente gerenciado.
- **O que foi feito:** Foi criada a fila 'pedidos-a-processar'. Quando o cliente envia uma requisição POST/pedidos para a API Flask, a API não processa o pedido imediatamente: ela apenas publica o evento no SQS e responde informando que o pedido que o navegador ou aplicativo fez ao servidor funcionou (HTTP 200) ao cliente.
- **Por que/Para que:** Desacoplamento de sistemas. Se o sistema de backend responsável por processar o pedido estiver lento ou fora do ar, o cliente não fica travado aguardando e a requisição não é perdida; ela aguarda com segurança na fila.
- **Arquivo:**
 - sqs.tf

```

resource "aws_sqs_queue" "pedidos_queue" {
  name = "pedidos-a-processar"
  delay_seconds = 0
  max_message_size = 262144
  message_retention_seconds = 345600
  receive_wait_time_seconds = 0
}

```

Passo 6: Computação *Serverless* e Orientada a Eventos (AWS Lambda)

- **Conceito:** O AWS Lambda permite executar código sem gerenciar servidores. O custo está atrelado apenas pelo tempo de execução em milissegundos.

TAREFA

- **O que foi feito:** Foi configurada uma função Lambda em Python que escuta a fila SQS através de um gatilho (*Event Source Mapping*). Sempre que uma nova mensagem entra na fila, a Lambda é invocada automaticamente para consumir a mensagem.
- **Por que/Para que:** Alta eficiência e baixo custo. Não há um servidor rodando ininterruptamente para processar pedidos; o recurso computacional é criado sob demanda no exato instante em que a mensagem chega e destruído logo em seguida.
- **Arquivo:**
 - lambda.tf

```
resource "aws_iam_role" "lambda_role" {
  name = "capacita-irede-lambda-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "lambda.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "lambda_sqs_basic" {
  role       = aws_iam_role.lambda_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaSQSQueueExecutionRole"
}

data "archive_file" "lambda_zip" {
  type      = "zip"
  output_path = "${path.module}/lambda.zip"

  source {
    content = <<-PYTHON
    import json

    def lambda_handler(event, context):
        for record in event['Records']:
            payload = record['body']
            print(f"PROCESSAMENTO CONCLUÍDO -> Mensagem recebida do SQS: {payload}")
            return {'statusCode': 200}
    PYTHON
    filename = "index.py"
  }
}

resource "aws_lambda_function" "processa_pedidos" {
  filename      = data.archive_file.lambda_zip.output_path
  function_name = "processa-pedidos-lambda"
  role          = aws_iam_role.lambda_role.arn
  handler       = "index.lambda_handler"
  runtime       = "python3.9"
  source_code_hash = data.archive_file.lambda_zip.output_base64sha256
}

# Gatilho: Conecta a Fila SQS à Função Lambda
resource "aws_lambda_event_source_mapping" "sqs_trigger" {
  event_source_arn = aws_sqs_queue.pedidos_queue.arn
  function_name     = aws_lambda_function.processa_pedidos.arn
  batch_size        = 1
}
```

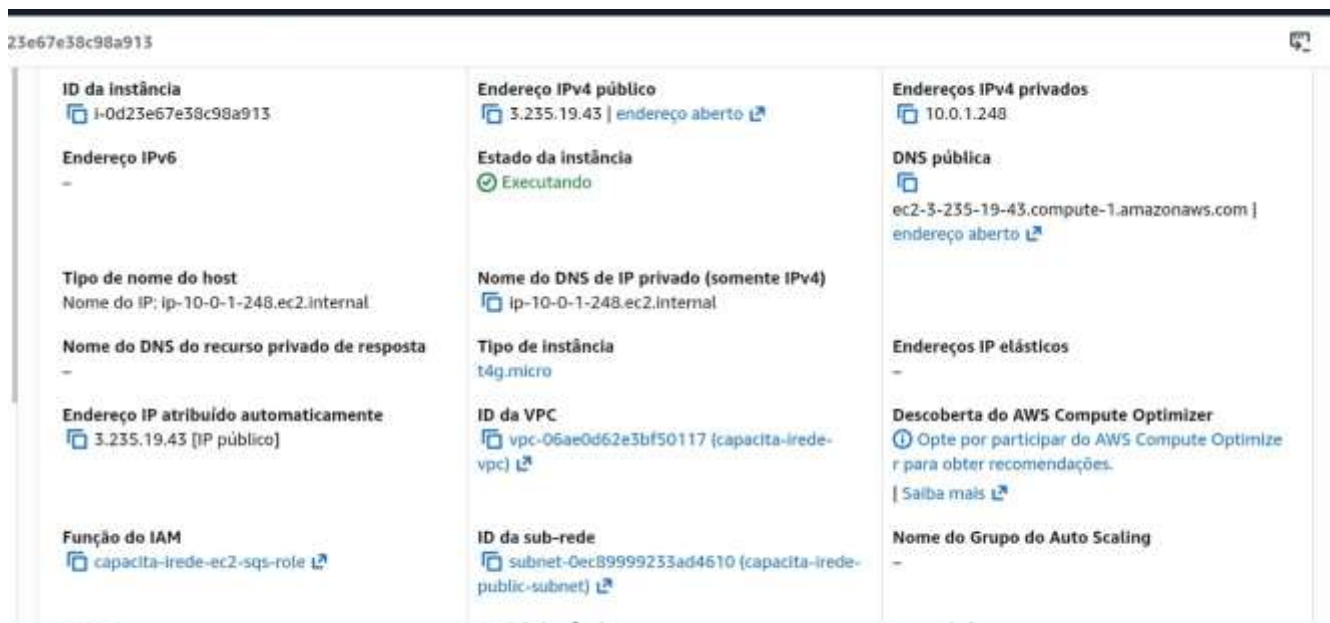
Passo 7: Observabilidade e Auditoria (AWS CloudWatch)

- **Conceito:** O Amazon CloudWatch é o serviço de monitoramento e gerenciamento de logs da AWS.
- **O que foi feito:** Foi configurada uma função Lambda para gravar cada processamento no grupo de logs `/aws/lambda/processa-pedidos-lambda`.
- **Por que/Para que:** Garantir a observabilidade. Através dos logs no CloudWatch, é possível auditar transações, identificar os pedidos processados e depurar eventuais falhas do sistema.
- **Arquivo:** Não necessita de arquivo específico. Quando a instrução `'print(...)'` é executada dentro da função Lambda, a AWS redireciona essa saída automaticamente para o CloudWatch Logs. Isso fornece rastro de auditoria para cada mensagem processada.

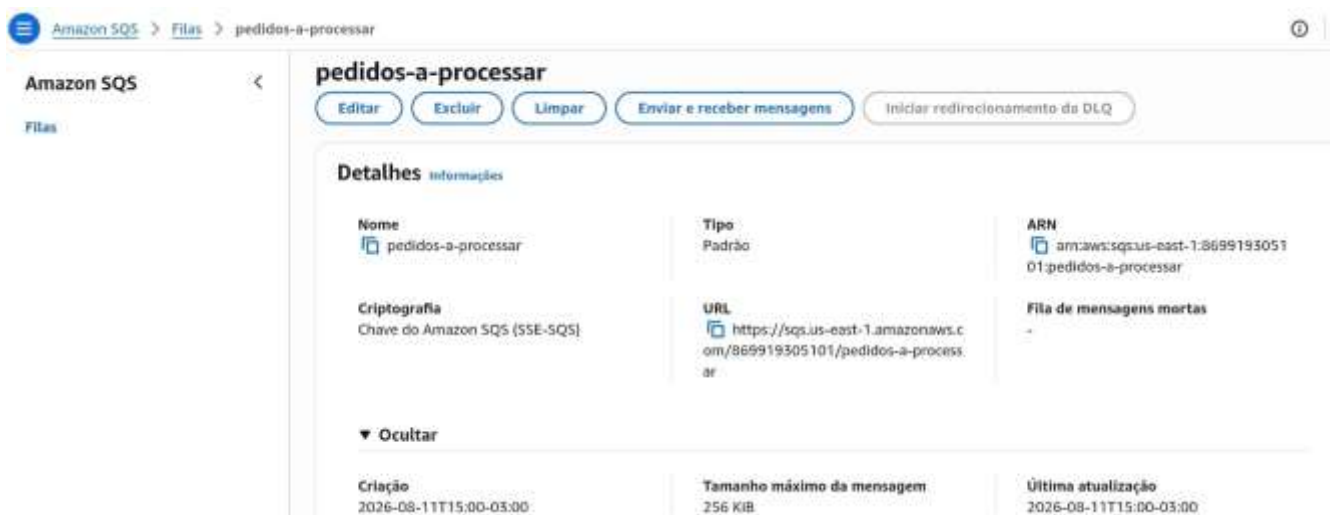
4. O Ciclo de Vida do Terraform na Prática

No Termux - aplicativo livre e de código aberto para Android que funciona como um emulador de terminal e ambiente Linux - foram utilizados os seguintes comandos no fluxo de trabalho IaC com Terraform:

1. **terraform init**: Baixa os provedores necessários (como o provedor da AWS) e inicializa o diretório do projeto.
2. **terraform validate**: Checa a sintaxe dos arquivos .tf para evitar erros antes da execução.
3. **terraform plan**: Compara o estado atual da AWS com o código local e exibe um relatório prévio com os recursos que serão criados, alterados ou destruídos.
4. **terraform apply -auto-approve**: Executa as chamadas de API da AWS para construir a infraestrutura declarada.



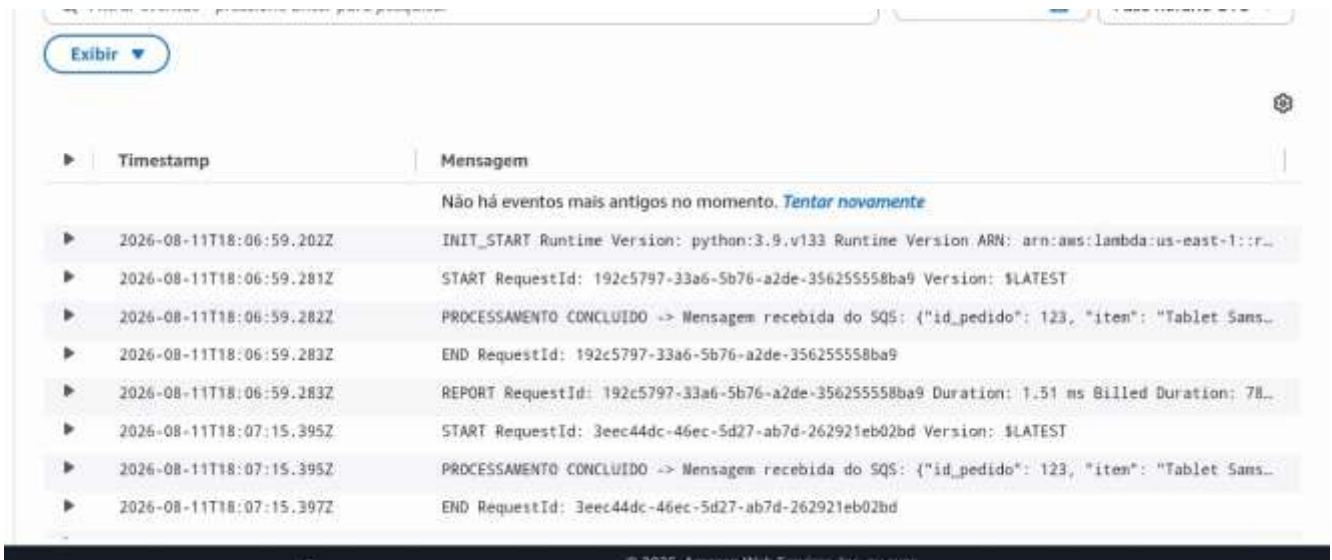
5. **curl -X POST ...** : Envia o pedido HTTP para a EC2 na porta 5000.



TAREFA

```
Apply complete! Resources: 13 added, 0 changed, 0 destroyed.  
Outputs:  
ec2_public_ip = "3.91.3.86"  
(.venv) root@localhost:~/projeto-capacita/terraform# curl http://3.91.3.86:5000/produtos  
[{"id":1,"nome":"Tablet Samsung S10 Lite","preco":2499.0}, {"id":2,"nome":"Capa Teclado Bluetooth","preco":349.9}]  
(.venv) root@localhost:~/projeto-capacita/terraform#
```

6. Verificação no CloudWatch: Demonstra o log do evento processado pela Lambda.



Timestamp	Mensagem
	Não há eventos mais antigos no momento. Tentar novamente
2026-08-11T18:06:59.202Z	INIT_START Runtime Version: python:3.9.v133 Runtime Version ARN: arn:aws:lambda:us-east-1::r...
2026-08-11T18:06:59.281Z	START RequestId: 192c5797-33a6-5b76-a2de-35625558ba9 Version: \$LATEST
2026-08-11T18:06:59.282Z	PROCESSAMENTO CONCLUÍDO -> Mensagem recebida do SQS: {"id_pedido": 123, "item": "Tablet Sans...
2026-08-11T18:06:59.283Z	END RequestId: 192c5797-33a6-5b76-a2de-35625558ba9
2026-08-11T18:06:59.283Z	REPORT RequestId: 192c5797-33a6-5b76-a2de-35625558ba9 Duration: 1.51 ms Billed Duration: 78...
2026-08-11T18:07:15.395Z	START RequestId: 3eec44dc-46ec-5d27-ab7d-262921eb02bd Version: \$LATEST
2026-08-11T18:07:15.395Z	PROCESSAMENTO CONCLUÍDO -> Mensagem recebida do SQS: {"id_pedido": 123, "item": "Tablet Sans...
2026-08-11T18:07:15.397Z	END RequestId: 3eec44dc-46ec-5d27-ab7d-262921eb02bd

7. `terraform destroy`: Remove todos dos recursos provisionados, garantindo a gestão de custos e o descarte limpo do ambiente.